

Oracle Database 11g: Administration

Support de cours - Module III

D50079EN10

Edition 1.0

July 2012

D53581



Plan du cours

- 1 **PRESENTATION GENERAL D'ORACLE**_____
- 2 **ARCHITECTURE D'UN SERVEUR ORACLE** _____
- 3 **REVISION SUR LE LANGAGE SQL** _____
- 4 **INSTALLATION ET CREATION D'UNE BASE DE DONNEES ORACLE**_____
- 5 **ADMINISTRATION D'UNE BASE DE DONNEES ORACLE**_____
- 6 **GESTION DES UTILISATEURS**_____
- 7 **GESTION DES OBJECTS** _____
- 8 **GESTION DU STOCKAGE D'UNE BASE DE DONNEES**_____
- 9 **UTILISATION DU RECOVERY MANAGER (RMAN)** _____
- 10 **SECURITE ET SURVEILLANCE DE LA BASE DE DONNEES**_____
- 11 **MINI PROJET**_____



Gestion des Objets

ORACLE

Copyright © 2007, Oracle. Tous droits réservés.

Introduction

Les objets sous Oracle sont les structures logiques qui font directement référence aux données de la base. Ils incluent des structures telles que des tables, des vues, des séquences, des procédures stockées, des synonymes, des index, des clusters et des liens de base de données.

1- Table

C'est une entité dont les informations sont stockées dans des colonnes et des lignes. Ces colonnes ont des noms, type de données et des tailles

Syntaxe :

```
CREATE TABLE nom_table (nom_col1 type_col1 [constraint], nom_col2 typ_col2 [constraint],...)  
[TABLESPACE nom_tbs]  
[STORAGE INITIAL taille K|M NEXT taille K|M]
```

Cette commande permet de créer une table dans le tablespace spécifié ou dans le tablespace par défaut (SYSTEM) avec les paramètres de stockage physique spécifiés par la clause STORAGE. taille est un nombre qui précise la taille initial en KO ou MO de la table ainsi que la taille suivante.

Exemple

```
CREATE TABLE nouveaux_clients  
(Code NUMBER (4) PRIMARY KEY, nom VARCHAR(10))  
TABLESPACE tbs_clients  
STORAGE INITIAL 2MO NEXT 2MO ;
```

2- Vue

C'est le résultat stocké d'une requête et elle n'a pas d'existence propre.

Les Vues permettent de renforcer la sécurité des données par masquage des lignes et des colonnes sensibles aux usagers non habilités.

Syntaxe :

```
CREATE [OR REPLACE] VIEW nom_vue  
AS SELECT col1, col2,coln  
FROM nom_table [nom_table2,...]  
WHERE [condition]
```

Exemple

```
CREATE VIEW vue_annuaire  
AS SELECT nom,no_tel FROM annuiare  
Where Ville ='COTONOU'
```

3- Index

Ils permettent d'accéder plus rapidement aux données et de gérer l'unicité des Clé Primaires. Sur une même table peuvent cohabiter plusieurs index et un index peut porter sur une ou plusieurs colonnes

Syntaxe :

```
CREATE INDEX nom_index  
ON nom_table (col1, col2,coln)  
[ Tablespace nom_tablespace]
```

Exemple

```
CREATE INDEX idx_clients  
ON clients (no_client)  
TABLESPACE tbs_idx
```

4- Synonyme

Un synonyme est un autre nom qu'on donne à un objet afin de le référencer différemment. Il est possible d'affecter un synonyme à un objet de type table, vue, séquence, synonyme.

Syntaxe :

```
CREATE [PUBLIC] SYNONYM nom_synonym  
FOR nom_objet
```

Exemple

```
CREATE SYNONYM EMPLOYE  
FOR scott.emp
```

5- Sequence

Les séquences servent en oracle à représenter les données numériques incrémenté automatiquement. Ils permettent de créer des clés neutres, de gérer l'attribution des valeurs des clés primaires et enfin d'alléger les applicatifs. Une séquence est toujours de type numérique.

Syntaxe

```
CREATE SEQUENCE nom_sequence  
INCREMENT BY n  
[START WITH n]  
[MAXVALUE n | NOMAXVALUE]  
[CYCLE|NOCYCLE]
```

Exemple

```
CREATE SEQUENCE seq_emp  
INCREMENT by 1  
START WITH 1000  
NOMAXVALUE
```

Remarque:

Les Séquences sont utilisées avec les mots clés suivants : **currval** et **nextval**

- Select seq_empno.currval from dual ; (donne la valeur courante de la sequence)
- Select seq_empno.nextval from dual ; (donne la valeur suivante de la sequence)

6- Les DBlinks (Database links)

Les database links sont des références à des bases de données distantes. Ceci permet au sein d'une même session SQL d'accéder à différents objets de bases réparties sur le réseau. Une description des database links se trouve dans la vue USER_DB_LINKS du dictionnaire.

Syntaxe: CREATE DATABASE LINK <DBlink_name> CONNECT TO <user> IDENTIFIED BY <pwd> USING <'BD_NAME'>

Exemple: Select * from balance@ DBlink_name

7- Procédure stockée

C'est un ensemble d'ordre SQL et PL/SQL (programme) qui réalise une tâche.

Introduction a Oracle PL/SQL

PL/SQL (pour PROCEDURAL LANGUAGE/SQL) est un langage procédural d'Oracle corporation étendant SQL. Il permet de combiner les avantages d'un langage de programmation classique avec les possibilités de manipulation de données offertes par SQL.

1 Structure d'un programme PL/SQL

La structure de base d'un programme PL/SQL est celle de bloc (possiblement imbriquée).

Il a généralement la forme suivante:

DECLARE

/ section de declaration */*

BEGIN

/ corps du bloc de programme*

*Il s'agit de la seule zone dont la présence est obligatoire */*

EXCEPTION

/ gestion des exceptions */*

END;

Le corps du programme (entre le BEGIN et le END) contient des instructions

PL/SQL (assignements, boucles, appel de procédure) ainsi que des instructions SQL.

Il s'agit de la seule partie qui soit obligatoire. Les deux autres zones, dites zone de

Déclaration et zone de gestion des exceptions sont facultatives. Les seuls ordres

SQL que l'on peut trouver dans un bloc PL/SQL sont: **SELECT, INSERT, UPDATE, DELETE**. Les autres types d'instructions (par exemple CREATE, DROP, ALTER) ne

Peuvent se trouver qu'à l'extérieur d'un tel bloc.

Chaque instruction se termine par un ";".

Le PL/SQL ne se soucie pas de la casse (majuscule vs. minuscule). On peut inclure des commentaires par --- (en début de chaque ligne commentée) ou par /*....*/ (pour délimiter des blocs de commentaires).

2 Variables et types

Un nom de variable en PL/SQL comporte au plus 30 caractères. Toutes les variables ont un type On trouve trois sortes de types de variables en PL/SQL. A savoir :

- un des types utilisés en SQL pour les colonnes de tables.
- un type particulier au PL/SQL.
- un type faisant référence à celui d'une (suite de) colonne(s) d'une table.

Les types autorisés dans PL/SQL sont nombreux. On retiendra principalement:

- Pour les types numériques : **INTEGER**, **NUMBER** (précision de 38 chiffres par défaut), **NUMBER(x)** (nombres avec x chiffres de précision), **NUMBER(x,y)** (nombres avec x chiffres de précision dont y après la virgule).
- Pour les types alphanumériques : **CHAR(x)** (chaîne de caractère de longueur fixe x), **VARCHAR(x)** (chaîne de caractère de longueur variable jusqu'à x), **VARCHAR2** (idem que précédent excepte que ce type supporte de plus longues chaînes et que l'on est pas obligé de spécifier sa longueur maximale). PL/SQL permet aussi de manipuler des dates (type **DATE**) sous différents formats.

Une déclaration pourrait donc contenir les informations suivantes :

DECLARE

n NUMBER;

mot VARCHAR(20) ;

mot2 VARCHAR2 ;

Une autre spécificité du PL/SQL est qu'il permet d'assigner comme type à une Variable celui d'un champ d'une table (par l'opérateur %TYPE) ou d'une ligne entière (Opérateur %ROWTYPE). Dans la déclaration suivante :

DECLARE

nom emp.name%TYPE;

employe emp%ROWTYPE;

La variable nom est définie comme étant du type de la colonne name de la table emp (qui doit exister au préalable). De même, employé est un vecteur du type d'une ligne de la table emp. A supposer que cette dernière ait trois champs numéro, name, age de type respectifs NUMBER, VARCHAR, INTEGER, la variable employe disposera de trois composantes :employe.numero, employe.name, employe.age, de même types que ceux de la table.

Soit ce premier programme

DECLARE

A NUMBER;

b NUMBER;

BEGIN

a:=a+b;

END;

(noter au passage l'instruction d'affectation "a:=a+b")

Un deuxième programme (incluant une requete SQL) :

```
DECLARE
```

```
a    NUMBER;
```

```
BEGIN
```

```
SELECT numero INTO a FROM emp
```

```
WHERE noemp='21';
```

```
END;
```

Ce dernier exemple donne comme valeur `a la variable a le resultat de la requete (qui doit être du meme type). Il est impératif que la requete ne renvoie qu'un et un seul résultat (c'est `a dire qu'il n'y ait qu'un seul employé numero 21). Autrement, une erreur se produit.

3 Opérateurs

PL/SQL supporte les operateurs suivants :

- Arithmetique : +, - *, /, ** (exponentielle)
- Concatenation : ||
- Parentheses (controle des priorites entre operations):
 - Affectation: :=
 - Comparaison : =, !=, <, >, <=, >=, IS NULL, LIKE, BETWEEN, IN
 - Logique : AND, OR, NOT

EXERCICE 1:

Soit le table: **EMP (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO)**

Ecrire un programme pl/sql permettant d'insérer un nouvel employé avec comme numéro

matricule une séquence et comme salaire la moyenne des salaires du département (deptno) 30

4 Structures de contrôle

Comme n'importe quel langage procédural, PL/SQL possède un certain nombre de structures de contrôles évoluées comme les branchements conditionnels et les boucles.

4.1 Les branchements conditionnels

Syntaxe :

IF <condition> THEN

commandes;

[ELSEIF <condition> THEN

commandes;]

[ELSE

commandes;]

END IF;

- exemple :

IF nomEmploye='TOTO' THEN

salaire:=salaire*2;

ELSEIF salaire>10000 THEN

salaire:=salaire/2;

ELSE

salaire:=salaire*3;

END IF;

4.2 Boucles

PL/SQL admet trois sortes de boucles. La première est une boucle potentiellement infinie :

LOOP

commandes;

END LOOP;

Au moins une des instructions du corps de la boucle doit être une instruction de sortie :

EXIT WHEN <condition>;

Dès que la condition devient vraie (si elle le devient...), on sort de la boucle.

Le deuxième type de boucle permet de répéter un nombre défini de fois un même traitement :

FOR <compteur> IN [REVERSE] <limite_inf> .. <limite_sup>

commandes;

END LOOP;

Enfin, le troisième type de boucle permet la sortie selon une condition prédéfinie.

WHILE <condition> LOOP

commandes;

END LOOP;

Toutes ces structures de contrôles sont évidemment imbriquables les unes dans les autres. Voici un même exemple qui traite de trois manières différentes suivant le type de boucle choisi.

Premier exemple :

DECLARE

x NUMBER(3):=1;

BEGIN

LOOP

INSERT INTO employe(noemp, nomemp, job, nodept)

VALUES (x, 'TOTO', 'PROGRAMMEUR', 1);

x:=x+1;

EXIT WHEN x>=100

END LOOP;

END;

Deuxieme exemple :

DECLARE

x NUMBER(3);

BEGIN

FOR x IN 1..100

INSERT INTO employe (noemp, nomemp, job, nodept)

VALUES (x, 'TOTO', 'PROGRAMMEUR', 1);

END LOOP;

END;

Troisieme exemple :

DECLARE

x NUMBER(3):=1;

BEGIN

WHILE x<=100 LOOP

INSERT INTO employe(noemp, nomemp, job, nodept)

VALUES (x, 'TOTO', 'PROGRAMMEUR', 1);

x:=x+1;

END LOOP;

END;

Exercice2:

Repeter l'exemple precedent 10 fois en mettant les employes a code paire dans la premiere table et les employés a code impair dans une seconde table

5- Curseurs

Jusqu'à présent, nous avons vu comment récupérer le résultat de requêtes SQL dans des variables PL/SQL lorsque ce résultat ne comporte au plus qu'une seule ligne.

Bien évidemment, cette situation est loin de représenter le cas général : les requêtes renvoient très souvent un nombre important et non prévisible de lignes. Dans la plupart des langages procéduraux au dessus de SQL, on introduit une notion de "curseur" pour récupérer (et exploiter) les résultats de requêtes.

Un curseur est une variable dynamique qui prend pour valeur le résultat d'une requête. La méthode pour utiliser un curseur est invariable. En premier lieu, celui-ci doit être défini (dans la zone de déclaration). Exemple :

CURSOR empCur IS SELECT * FROM emp;

Le curseur de nom empCur est chargée dans cet exemple de récupérer le résultat de la requête qui suit. Il peut alors être ouvert lorsqu'on souhaite l'utiliser (dans le corps d'un bloc) :

OPEN empCur;

Pour récupérer les tuples successifs de la requête, on utilise l'instruction :

FETCH empCur INTO employeVar;

La première fois, c'est le premier tuple du résultat de la requête qui est affectée comme valeur à la variable employeVar (qui devra être définie de façon adéquate).

À chaque exécution de l'instruction, un nouveau tuple résultat est chargé dans employeVar. Enfin, lorsque le traitement sur le résultat de la requête est terminée, on ferme le curseur.

CLOSE empCur;

L'exemple suivant sélectionne l'ensemble des employés dont le salaire ne dépasse pas 6000 francs et les augmente de 500 francs.

```
DECLARE

CURSOR SalCur IS

SELECT * FROM EMP WHERE SAL<6000.00;

employe EMP%ROWTYPE;

BEGIN

OPEN SalCur;

LOOP

FETCH SalCur INTO employe;

UPDATE EMP

SET SAL=6500.00      WHERE EMPNO=employe.empno;

EXIT WHEN SalCur%NOTFOUND;

END LOOP;

END;
```

Lorsqu'on souhaite parcourir un curseur dans une boucle pour effectuer un traitement, on peut simplifier l'utilisation de ceux-ci. Le programme suivant, parfaitement l'égal, est équivalent au précédent.

```

DECLARE

CURSOR SalCur IS

SELECT * FROM EMP WHERE SAL<6000.00;

employe EMP%ROWTYPE;

BEGIN

FOR employe IN SalCur

LOOP

UPDATE EMP

SET SAL=6500.0 WHERE EMPNO=employe.empno;

END LOOP;

END;

```

Dans cette boucle, l'ouverture, la fermeture et l'incrémentation du curseur SalCur sont implicites et n'ont pas besoin d'être spécifiques.

Un certain nombre d'informations sur l'état d'un curseur sont exploitables à

l'aide d'attributs prédéfinis : l'attribut **%FOUND** renvoie vrai si le dernier FETCH a

Bien ramené un tuple. L'attribut **%NOTFOUND**, permet de décider si on est arrivé en fin de

curseur. **%ROWCOUNT** renvoie le nombre de lignes ramenées du curseur au moment de

l'interrogation. Enfin **%ISOPEN** permet de déterminer si le curseur est ouvert.

EXERCICE 2:

1- Créer une table **AGREGAT**(deptno,deptname, tot_salaire,tot_comm,nbre_employe)

Ecrire un programme pl/sql permettant d'enrichir cette table avec un curseur.

6- La gestion des exceptions

PL/SQL permet de définir dans une zone particulière (de gestion d'exception), l'attitude que le programme doit avoir lorsque certaines erreurs définies ou prédéfinies se produisent.

Un certain nombre d'exceptions sont prédefinies sous Oracle. Citons, pour les plus fréquentes : `NO DATA FOUND` (devient vrai dès qu'une requête renvoie un résultat vide), `TOO MANY ROWS` (requête renvoie plus de lignes qu'escompte), `CURSOR ALREADY OPEN` (curseur déjà ouvert), `INVALID CURSOR` (curseur invalide)...

L'utilisateur peut définir ses propres exceptions. Dans ce cas, il doit définir celles-ci dans la zone de déclaration. Exemple :

excpt1 EXCEPTION

Puis, cette exception est levée quelque part dans le programme (après un test non concluant, par exemple), par l'instruction :

RAISE excpt1

Enfin, dans la zone d'exception un traitement est affecté à chaque exception possible (définie ou prédéfinie) :

EXCEPTION

WHEN <exception1> [OR <exception2> OR ...] THEN <instructions>

WHEN <exception3> [OR <exception2> OR ...] THEN <instructions>

WHEN OTHERS THEN <instructions>

END;

Evidemment, un seul traitement d'exception peut se produire avant la sortie du bloc.

Autre Exemple : **Calcul du prix TTC des produits et recopie dans la table PRODTTC**

```
DECLARE
nbp NUMBER(3);
aucun_produit EXCEPTION;
CURSOR calcul IS
SELECT numprod, prixuni*1.206 prixttc
FROM produit;
tuple calcul%ROWTYPE;
BEGIN
-- Comptage des produits
SELECT COUNT(*) INTO nbp FROM produit;
-- Test « il existe des produits » ou pas ?
IF nbp = 0 THEN
RAISE aucun_produit;
END IF;
-- Recopie des valeurs dans la table prodttc
FOR tuple IN calcul LOOP
INSERT INTO prodttc VALUES
(tuple.numprod, tuple.prixttc);
END LOOP;
-- Validation de la transaction
COMMIT;

EXCEPTION
WHEN aucun_produit THEN
RAISE_APPLICATION_ERROR (-20501, 'Erreur : table client vide');

END;
```

7- Procédures et fonctions

Il est possible de créer des procédures et des fonctions comme dans n'importe quel langage de programmation classique. La syntaxe de création d'une procédure est la suivante :

CREATE OR REPLACE PROCEDURE <nom>

AS

<Zone de déclaration de variables>

BEGIN

<corps de la procédure>

EXCEPTION

<Traitement des exceptions>

END;

A noter que toute fonction ou procédure créée devient un objet part entière de la base (comme une table ou une vue, par exemple). Elle est souvent appelée "procédure stockée". Elle est donc, entre autres, sensible à la notion de droit : son créateur peut décider par exemple d'en permettre l'utilisation à d'autres utilisateurs. Elle est aussi appelable depuis n'importe quel bloc PL/SQL avec la commande :

EXECUTE [NOM_PROCEDURE] ;

Example

```
CREATE OR REPLACE PROCEDURE Insert_T
AS
x NUMBER ;
y NUMBER ;
Matricule NUMBER ;
Salaire NUMBER ;
BEGIN
Select empno into x from scott.emp where ename='JONES' ;
Matricule:=x+1;
Select sal into y from scott.emp where ename='JONES' ;
Salaire := y+ (y*20/100) ;
INSERT INTO T VALUES (matricule, salaire);
Commit ;
END;
```

On peut aussi créer des fonctions. Le principe est le meme, l'en-tête devient :

```
CREATE OR REPLACE FUNCTION <nom> [(parametres)] RETURN <type du resultat>  
IS  
BEGIN  
  <Instructions>;  
  Return <var>;  
END;
```

Une instruction **RETURN** <expression> devra se trouver dans le corps pour specifier

Le type du resultat est renvoyé.

Example

```
CREATE OR REPLACE FUNCTION module (a NUMBER, b NUMBER) RETURN NUMBER IS  
BEGIN  
  IF a < b THEN  
    RETURN a ;  
  ELSE  
    RETURN (b-a);  
  END IF ;  
END;
```

Les procédures et fonctions PL/SQL supportent assez bien la surcharge (i.e. co-existence de procédures de même nom ayant des listes de parametres differentes).

C'est le systeme qui, au moment de l'appel, decidera, en fonction du nombre d'arguments et de leur types, quelle est la bonne procedure a appeler.

On peut detruire une procedure ou une fonction par:

```
DROP PROCEDURE <nom de procedure>
```

8- Les déclencheurs (“triggers”)

Les déclencheurs ou ”triggers” sont des séquences d’actions définies par le programmeur qui se déclenchent, non pas sur un appel, mais directement quand un événement particulier (spécifiée lors de la définition du trigger) sur une ou plusieurs tables se produit.

Un trigger sera un objet stocké (comme une table ou une procédure) La syntaxe est la suivante:

```
CREATE [OR REPLACE] TRIGGER <nom>  
{BEFORE|AFTER} {INSERT|DELETE|UPDATE} ON <nom de table>  
[FOR EACH ROW [WHEN (<condition>)]]  
<corps du trigger>
```

Un trigger se déclenche avant ou après (BEFORE|AFTER) une insertion, suppression ou mise à jour (INSERT|DELETE|UPDATE) sur une table (à noter que l’on peut exprimer des conditions plus complexes avec le OR: INSERT OR DELETE ...).

L’option FOR EACH ROW [WHEN (<condition>)] fait s’exécuter le trigger à chaque modification d’une ligne de la table spécifiée (on dit que le trigger est de ”niveau ligne”). En l’absence de cette option, le trigger est exécutée une seule fois (”niveau table”). Soit l’exemple suivant :

les procédures et fonctions sont des objets stockés. On peut donc les détruire par des instructions similaires aux instructions de destructions de tables...

```
CREATE TABLE T1 (num1 INTEGER, num2 INTEGER);
```

```
CREATE TABLE T2 (num3 INTEGER, num4 INTEGER);
```

```
CREATE TRIGGER inverse
```

```
AFTER INSERT ON T1
```

```
FOR EACH ROW WHEN (NEW.num1 <=3)
```

```
BEGIN
```

```
INSERT INTO T2 VALUES(:NEW.num2, :NEW.num1);
```

```
END inverse;
```

Ce trigger va, en cas d'insertion d'un tuple dans T 1 dont la premiere coordonnee est inferieure `a 3, inserer le tuple inverse dans T 2. Les prefixes **NEW** et **OLD** (en cas de UPDATE ou de DELETE) vont permettre de faire reference aux valeurs des colonnes apres et avant les modifications dans la table. Ils sont utilisés sous la forme NEW.num dans la condition du trigger et sous la forme : NEW.num dans le corps.

Un trigger peut etre activée ou desactivée par les instructions :

```
ALTER TRIGGER <nom> {ENABLE|DISABLE};
```

et détruit par :

```
DROP TRIGGER <nom>.
```

9 Quelques remarques

9.1 Affichage

PL/SQL n'est pas un langage avec des fonctionnalités d'entrées sorties évoluées (ce n'est pas son but!). Toutefois, on peut imprimer des messages et des valeurs de variables de plusieurs manières différentes. Le plus pratique est de faire appel à un package prédéfini : DBMS output.

En premier lieu, taper au niveau de la ligne de commande :

SET SERVEROUTPUT ON

Pour afficher, on utilise alors la commande suivante :

DBMS_OUTPUT.PUT_LINE ('Au revoir' || nom || ' a bientôt');

Si on suppose que nom est une variable dont la valeur est "toto", l'instruction

Affichera : Au revoir toto a bientôt.

9.2 Debugger

Pour chaque objet créée (procédure, fonction, trigger...), en cas d'erreur de compilation, on peut voir ces erreurs en tapant (si "essai" est un nom de procédure)

SHOW ERRORS PROCEDURE essai;

EXERCICE1

L'entreprise employant les personnes de la table EMP de l'exo n° 1(cours) est délocalisée des États-Unis en France (si, si). Il est donc nécessaire de convertir leur salaire et leur commission en francs (pour simplifier,on admettra 1 USD = 6,5 FF). Tous les employés voient également augmenter leur salaire de 25 % après conversion.

- 1) Créer une nouvelle table vide EMP_FR de même structure que EMP a partir d'une seule requête.
- 2) Écrire un programme PL/SQL permettant de recopier tous les tuples de la table EMP dans la table EMP_FR en effectuant au passage les opérations nécessaires sur le salaire et la commission. Traiter le cas où la table EMP est vide comme une exception.

Exercice 2

Soit une table quelconque TABL, dont la clé primaire CLENUM est numérique. Définir un trigger en insertion permettant d'implémenter une numérotation automatique de la clé. Le premier numéro doit être 1.

Exercice 3

Soit le schéma relationnel d'une agence bancaire régionale.

CLIENT (NUMCL, NOM, PRENOM, ADR, CodeVILLE, SALAIRE, CONJOINT)

Ville (CodeVILLE,nomville,departement)

Écrire un trigger en insertion permettant de contrôler les contraintes suivantes :

- La Vile dans laquelle habite le client doit être Cotonou,calavi,ouidah,ou Porto-Novo;
- Le nom du conjoint doit être le même que celui du client.

ORACLE®